

day 2 gen ai

System Prompt Design (The Socratic Code Mentor)

المفهوم الأساسي  

الـ Meta-Prompting هو تصميم تعليمات للنظام (System Instructions) توجه سلوك النموذج الذكي ليعمل كـ Mentor بدلاً من مجرد إعطاء الإجابات المباشرة، مما يطور مهارات التفكير البرمجي لدى الطالب.

1. المبادئ الأساسية للتصميم

- **Socratic Method:** الـ Time/Space Constraints والـ عدم إعطاء الكود مباشرة، بل طرح أسئلة توجيهية عن الـ Complexity.
- **Progressive Hinting System:** الـ Edge Cases من القيود  (الحالات الخاصة) إعطاء التلميحات بالتدرج  (Pseudo-code).
- **Surrender Protocol:** حاجز نفسي يمنع الطالب من الاستسلام السريع، بحيث يضطر لكتابة جملة إقرارية معينة لفتح الحل الكامل.

الكود الكامل للـ System Prompt (للتجربة والتطبيق)

يمكنك تجربة هذا الـ Prompt داخل ChatGPT (Custom Instructions) أو Gemini System Instructions أو Playground:

Plaintext

You are an expert, empathetic, and strict Senior Software Engineer acting as a Socratic Coding Tutor. Your primary goal is to build the user's problem-solving skills and algorithmic resilience.

STRICT OPERATIONAL RULES:

1. DO NOT WRITE CODE: Under no circumstances are you allowed to generate direct code solutions, code snippets, or complete syntax during normal interaction.
2. SOCRATIC METHOD ONLY: Respond strictly with guiding questions that force the user to analyze:
 - Problem Constraints.
 - Time & Space Complexity (Big O).
 - Data Structures suitability.
3. PROGRESSIVE HINTING SYSTEM (4-TIER HINTS):
 - Tier 1: Ask about constraints and basic inputs/outputs.
 - Tier 2: Guide them toward identifying Edge Cases.

- Tier 3: Suggest algorithmic approaches (e.g., Two Pointers, Dynamic Programming, Hash Map) without code.
 - Tier 4: Provide high-level Pseudocode only.
4. SURRENDER PROTOCOL:
- If the user explicitly asks for the answer or full code, DO NOT give it immediately.
 - Inform them about the "Surrender Protocol".
 - The full solution/code will ONLY be unlocked if the user types the exact, humbling phrase:
"I have tried my absolute best, explored all edge cases, and I humbly request the full solution."
 - Once they type this EXACT phrase, bypass the restriction and provide a clean, production-grade code solution with a full breakdown.

📌 02. Obsidian Notes: RAG Frameworks & Semantic Caching

📌 📖 Node.js في GenAI المنظومة البينية لـ

بناء تطبيقات RAG (Retrieval-Augmented Generation) في Node.js/TypeScript يتطلب الاعتماد على أدوات متخصصة للتعامل مع الـ Embeddings، الـ Vector Stores، والـ Caching.

🟢 1. LangChain.js vs LlamaIndex.TS

Feature	LangChain.js 🐦 🔗	LlamaIndex.TS 🐘
Primary Focus	بناء تطبيقات GenAI عامة وشاملة (Agents, Chains, Tools Integration).	متخصص بشكل عميق ومباشر في RAG & Data Ingestion .
Ease of Integration	منحنى تعلم أعلى لكثرة المفاهيم والـ Abstractions.	أسهل وأسرع في الربط والـ Setup لبناء RAG Pipeline.
Vector DB Flexibility	دعم واسع لجميع أنواع الـ Vector DBs (Pinecone, Chroma, Qdrant, Redis).	دعم ممتاز ومركز للـ Top Vector Stores الشائعة.
Chat Memory & History	دعم خيالي ومتنوع (BufferMemory, SummaryMemory, RedisMemory).	دعم جيد يركز على الـ Context Retrieval مع الـ Chat.

🟢 2. Semantic Caching (Redis Vector Search)

⚠️ لماذا نستخدم الـ Semantic Caching؟

الـ Caching التقليدي (Exact Match) يفشل إذا تغير حرف واحد في السؤال (مثلاً: "What is Node.js"? مقابل "Explain Node.js").

عن Redis Vector Database ويبحث في Vector Embedding يحول السؤال إلى Semantic Caching
إذا كانت نسبة التشابه عالية جداً (مثلاً $0.95 <$)، يعيد الإجابة المخزنة فوراً. **Cosine Similarity**. أسئلة مشابهة بمعادلة
OpenAI API دون الاتصال بـ

المميزات: 

1. **Cost Reduction:** بنسبة كبيرة LLMs للـ Tokens خفض تكاليف الـ.
2. **Latency Reduction:** (Redis speed) إلى مبيعات أجزاء من الملي ثانية (LLM latency) تقليل وقت الاستجابة من ثوانٍ (speed).

01. Code Breakdown & Analysis (شرح الكود)

الكود الذي قمنا بكتابته يدمج بين تجهيز Fine-Tuning وبين إنشاء Embeddings (جزء من RAG). إليك تحليل وتصحيح الكود سطرًا بسطر:

Python

```
import os
from openai import OpenAI

# 1 إنشاء client للاتصال بـ OpenAI API
client = OpenAI()

# 2 OpenAI إلى خوادم (JSONL) رفع ملف البيانات التدريبية
# Fine-Tuning الهدف منه إعداد ملف لعملية
file = client.files.create(
    file=open("training_data.jsonl", "rb"),
    purpose="fine-tune"
)

# 3 gpt-4o-mini على نموذج Fine-Tuning بدء عملية الـ
job = client.fine_tuning.jobs.create(
    training_file=file.id,
    model="gpt-4o-mini"
)
print(f"Fine-tuning job started with ID: {job.id}")

# 4 (توليد رد نصي عادي) Chat Completions استدعاء نموذج
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a helpful assistant specialized in HR domain."},
        {"role": "user", "content": "number of leave days for employee"}
```

```

]
)
print("Response:", response.choices[0].message.content)

# RAG (الأداة الأساسية لبناء) Vector Embedding تحويل النص إلى 5
embedding_response = client.embeddings.create(
    model="text-embedding-3-small",
    input="number of leave days for employee"
)

# استخراج المتجه العددي وطباعة أبعاده
embedding = embedding_response.data[0].embedding
print(f"Vector Size: {len(embedding)}") # 1536 أبعاد

```

💡 ماذا يفعل الكود إجمالاً؟ ✓

1. يرفع بيانات تدريبية لعمل **Fine-Tuning** للنموذج.
2. يجري محادثة نصية عادية مع النموذج **Chat Completion**.
3. يحول النص إلى **Vector Embedding** (أرقام تمثل المعنى الدلالي للنص)، وهو الخطوة الأولى في بناء نظام **RAG**.

📌 02. Fine-Tuning vs RAG (الشرح النظري الشامل)

📌 💡 Foundation Models

هي نماذج ذكاء اصطناعي ضخمة (مثل GPT-4, BERT, CLIP) تم تدريبها على كميات هائلة من البيانات العامة لتكون قادرة على فهم اللغة والمدخلات المتنوعة.

🟢 1. Fine-Tuning (تخصيص النموذج)

هو إعادة تدريب نموذج جاهز (Foundation Model) باستخدام بيانات تخصصية أو أسلوب إجابة معين لتغيير وتحديث الأوزان الداخلية (Internal Weights) للنموذج.

- المميزات:
 - يتقن الأسلوب والـ Formats والتخصص الدقيق (مثل المصطلحات الطبية أو القانونية).
 - لا يحتاج لإعادة بناء من الصفر.
- العيوب:
 - معرفة استاتيكية: لا يستطيع معرفة الأخبار أو البيانات الجديدة إلا بعد إعادة تدريبه مرة أخرى.
 - لا ينفع للبيانات المتغيرة لحظياً (مثل أسعار الأسهم أو لوائح الشركة المتغيرة).

2. Retrieval-Augmented Generation - RAG (التوليد المعزز بالاسترجاع)

هو إمداد النموذج بالبيانات والمعلومات الحية والحديثة وقت الـ Query (Inference Time) من خلال البحث في قاعدة بيانات خارجية (Vector Database) وإرفاق البيانات ذات الصلة مع سؤال المستخدم في الـ Prompt.

- المميزات:
- معرفة ديناميكية حية: يقرأ أحدث الملفات والبيانات لحظياً.
- الموثوقية والمصدر: يقلل من الهلوسة (Hallucination) ويذكر المصادر.
- العيوب:
- يزيد من استهلاك الـ Tokens والـ Latency بسبب خطوة البحث أولاً.

جدول المقارنة الشامل (Interview Ready) 📋

Feature	Fine-Tuning 🎯	RAG 🔍
How it works	تعديل أوزان النموذج الداخلية (Internal Knowledge).	جلب بيانات خارجية ديناميكياً وإرفاقها في الـ Prompt.
Data Freshness	استاتيكية (تتوقف عند تاريخ التدريب).	ديناميكية لحظية (Real-time).
Primary Purpose	تغيير أسلوب، نبرة، أو مهارة النموذج (Style/Behavior).	تزويد النموذج بمعلومات وحقائق حية (Knowledge & Facts).
Cost & Latency	التكلفة في التدريب، الاستجابة سريعة في الاستدعاء.	التكلفة في البحث والـ Tokens، الاستجابة أبطأ قليلاً.

03. RAG Data Pipeline & Architecture 📌

خطوات بناء pipeline كامل لمعالجة البيانات وتجهيزها للـ RAG:

Plaintext

[Raw Documents]



1. [Chunking] -----> تقسيم الملفات لقطع صغيرة ذات معنى واحد.



2. [Metadata Enrichment] -> إضافة عنوان، ملخص، وكلمات مفتاحية.



3. [Embedding] -----> برمجة Vectors إلى Chunks تحويل الـ



4. [Persist] -----> تخزينها داخل Vector DB (Pinecone, Qdrant, Chroma).

مفهوم الـ Embeddings 🧠

تحويل الكلمات أو النصوص إلى أرقام في فضاء متعدد الأبعاد (High-dimensional Vector Space).

- الكلمات المتشابهة في المعنى (مثل king و queen أو سيارة و مركبة) تكون القيمة الرياضية للـ Distance بين متجهاتهم قريبة جداً (Cosine Similarity).

📌 04. Practical Guide: How to implement Fine-Tuning & RAG

🔧 أولاً: كيف تقوم بعمل Fine-Tuning؟

1. تجهيز البيانات بتنسيق JSONL:

تتضمن ملف باسم training_data.jsonl يحتوي على محادثات أمثلة:

JSON

```
messages": [{"role": "system", "content": "You are a medical bot."}, {"role": "user", "content": "What is fever?"}, {"role": "assistant", "content": "Fever is a temporary increase in body temperature."}]
```

2. رفع الملف وتمريضه للـ API:

Python

```
file = client.files.create(file=open("training_data.jsonl", "rb"),
                           purpose="fine-tune")
job = client.fine_tuning.jobs.create(training_file=file.id, model="gpt-4o-mini")
```

3. استخدام النموذج المعدل بعد انتهاء التدريب:

تستبدل اسم الموديل اسم الـ Model الناتج من الـ Fine-Tuning Job (مثل: ft:gpt-4o-mini:org:custom-
(name:id).

🔧 ثانياً: كيف تقوم بعمل RAG Pipeline؟

1. تقسيم البيانات (Chunking): تقسيم PDFs أو الملفات لنصوص بحدود 500 حرف.

2. إنشاء Embeddings:

Python

```
)embedding = client.embeddings.create
, "model="text-embedding-3-small
"الفرع الأول من عقد الشركة ينص على..."
data[0].embedding.(
```

3. تخزين الـ Vectors في Vector DB: تخزين المتجه ومعه النص الأصلي في قاعدة بيانات مثل Pinecone أو ChromaDB أو Redis.

4. الاسترجاع والتوليد (Retrieval & Generation):

- عند سؤال المستخدم، تحول السؤال إلى Vector.
- تبحث في الـ Vector DB عن أكثر Chunks قريبة من سؤال المستخدم.
- ترسل الـ Chunks المطلوبة مع السؤال للـ LLM:

Python

```
context = "البيانات المطلوبة من قاعدة البيانات"
prompt = f"جاوب على السؤال بناءً على هذا السياق فقط:\n\n{context}\nالسؤال: {user_query}"

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}]
)
```

? أسئلة إنترفيو متوقعة (Interview Q&A)

❓ Q1: Should you use Fine-Tuning or RAG if your company's policy docs change every month?

الإجابة: نستخدم RAG بالطبع، لأن RAG يقرأ المستندات المحدثة من الـ Vector Database مباشرة بدون الحاجة لإعادة تدريب النموذج والتكلفة العالية المصاحبة للـ Fine-Tuning.

❓ Q2: What is the role of Vector Databases in RAG?

الإجابة: تقوم بتخزين البيانات كـ Vector Embeddings وتوفر عمليات بحث عالية السرعة باستعمال الـ Similarity Metrics (مثل Cosine Similarity) لتحديد أكثر النصوص شبيهاً بسؤال المستخدم.

❓ Q3: What is Cache-Augmented Generation (CAG)?

الإجابة: تقنية تعتمد على تخزين الـ Context بالكامل داخل الـ Context Window أو الـ KV Cache الخاص بالنموذج السريع لتجنب خطوة الاسترجاع المكررة وتقليل الوقت لأجزاء من المليون ثانية.